LAPORAN MILESTONE 1 TUGAS BESAR

IF2224 TEORI BAHASA FORMAL DAN AUTOMATA

Lexical Analysis

Disusun oleh:

JBC - J1B2C2

Samuelson D. Tanuraharja	13524001
Aufa Rienaldifaza Ahmad	13524027
Niko Samuel Simanjuntak	13524029
Aurelia Jennifer Gunawan	13524089
Reinsen Silitonga	13524093

PROGRAM STUDI TEKNIK INFORMATIKA

SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA

INSTITUT TEKNOLOGI BANDUNG

JL. GANESA 10, BANDUNG 40132

2026

DAFTAR ISI

DAFTAR ISI **1**
DAFTAR GAMBAR **2**
DAFTAR TABEL **3**
BAB 1
TEORI SINGKAT **4**
1.1. Lexical Analysis 4
1.2. Regular Expression pada Compiler 4
1.3. Mekanisme Pengenalan Token (Token Recognition) 5
1.4. Finite State Machine 5
1.5. Deterministic Finite Automata (DFA) 6
1.6. Implementasi DFA dalam Lexical Analysis 8
BAB 2
PERANCANGAN & IMPLEMENTASI **10**
2.1. Perancangan 10
2.2. Implementasi 18
2.2.1. Konsep Lexical Analyzer dan Peran DFA dalam Implementasi 18
2.2.2. Arsitektur dan Struktur Program 18
BAB 3
PENGUJIAN **20**
BAB 4
KESIMPULAN DAN SARAN **29**
4.1 Kesimpulan 29
4.2 Saran 29
BAB 5
LAMPIRAN **31**
5.1 Link Repository Github 31
5.2 Link Workspace Diagram Transisi DFA 31
5.3 Pembagian Tugas 31
5.4 Referensi 32

DAFTAR GAMBAR

Gambar 1. Diagram DFA 13

DAFTAR TABEL

Tabel 1. Keterangan State DFA 13
Tabel 2. Pengujian program 20
Tabel 3. Pembagian tugas 31

BAB 1
TEORI SINGKAT

1.1. *Lexical Analysis*

Lexical analysis merupakan langkah awal dari proses *compiling* pada suatu *compiler*. *Lexical analyzer* berfungsi untuk membaca *stream of characters* yang membentuk suatu program dan mengelompokkan karakter-karakter tersebut menjadi sejumlah rangkaian yang bermakna, disebut juga sebagai *lexemes*. Untuk setiap *lexeme*, *lexical analyzer* akan membuat *output* berupa *token*.

Token merupakan satuan terkecil dari sebuah program yang merepresentasikan sebuah simbol bermakna dari suatu kumpulan karakter. Dalam *lexical analysis*, sebuah token berbentuk

$$< tokenName, attributeValue >.$$

Misalnya, sebuah program yang memiliki masukan program

```
position = initial + rate * 60.
```

Lexical analyzer akan mengubah masing-masing kumpulan karakter menjadi

```
<id, 1> = <id, 2> + <id, 3> <*> <60>.
```

Dalam representasi ini, id merupakan simbol yang merepresentasikan *identifier* dan angka di sebelah kanan merupakan angka *table entry* yang terdapat pada *symbol table*.

1.2. *Regular Expression* pada *Compiler*

Untuk sebuah token, kita perlu memberikan *identifier* dengan memberikan nama pada sebuah *set* yang berisi huruf dan angka. Proses memberi notasi ini disebut sebagai *regular expression*. *Regular expression* digunakan untuk mendefinisikan *pattern* dari sebuah token. Secara formal, RE menyediakan notasi untuk mendefinisikan set karakter yang membentuk bahasa reguler. Menotasikan *set* ini dapat memanfaatkan operasi-operasi dasar *regular expression*, seperti *union, concatenation,* dan *closure*.

Untuk kemudahan proses pemberian notasi, dibentuk sebuah notasi *regular expression* sebagai berikut.

1. *One or more instances*, dinyatakan dengan *postfix* "+".

2. *Zero or one instance*, dinyatakan dengan *postfix* "?".
3. *Character classes*, misalnya sebuah ekspresi $a_1 \mid a_2 \mid \ldots \mid a_n$ yang a_i menyatakan simbol alfabet, dapat dinyatakan dengan $[a_1a_2\ldots a_n]$. Kita juga dapat menyatakan simbol sebelumnya dengan a_1–a_n.

Misalnya, kita ingin membuat *identifier* untuk *set* terdiri dari huruf alfabet kapital dan nonkapital, maka kita dapat menyatakannya dengan

```
[A-Za-z].
```

1.3. Mekanisme Pengenalan Token (*Token Recognition*)

Proses pengenalan token dilakukan dengan mencocokkan *stream* karakter terhadap sekumpulan regular definition yang telah ditentukan. Dalam tahap ini, lexical analyzer bertugas mencari *prefix* terpanjang dari input yang sesuai dengan salah satu pola (pattern). Karakter-karakter seperti spasi, tab, dan *newline* didefinisikan sebagai token *ws* (white space) yang dikenali yang akan memicu *scanner* untuk melakukan restart pencarian pada karakter berikutnya. Selain itu, terdapat aturan prioritas untuk reserved words; misalnya, meskipun kata *if* secara pola cocok dengan aturan identifier, ia harus tetap diidentifikasi sebagai token keyword if agar tidak terjadi ambiguitas semantik.

1.4. Finite State Machine

Finite State Machine (FSM) adalah model komputasi abstrak yang digunakan untuk mengenali pola dalam input. Dalam konteks *lexical analysis*, FSM berperan sebagai mekanisme implementasi dari regular expression untuk mengenali token. Sebuah FSM terdiri dari sejumlah state, sebuah state awal, satu atau lebih state akhir (accepting state), dan fungsi transisi yang menentukan perpindahan antar state berdasarkan karakter input yang dibaca.

Secara formal, sebuah Finite Automata didefinisikan sebagai 5-tuple: $M = (Q, \Sigma, \delta, q_0, F)$:

- **Q** adalah himpunan state yang terbatas,
- **Σ** adalah alfabet input (himpunan simbol yang dikenali),
- **δ** adalah fungsi transisi $\delta: Q \times \Sigma \rightarrow Q$ (untuk DFA) atau $\delta: Q \times \Sigma \rightarrow 2^Q$ (untuk NFA),
- $\mathbf{q_0} \in \mathbf{Q}$ adalah state awal,
- $\mathbf{F} \subseteq \mathbf{Q}$ adalah himpunan accepting state.

1.5. *Deterministic Finite Automata* (DFA)

Deterministic Finite Automata (DFA) merupakan salah satu model komputasi paling fundamental dalam teori bahasa formal dan automata. Menurut Hopcroft, Motwani, dan Ullman (2006), istilah "deterministic" merujuk pada fakta bahwa untuk setiap keadaan (*state*) dan setiap simbol masukan, terdapat tepat satu keadaan tujuan.

1.5.1. Definisi Formal DFA

Secara formal, sebuah Deterministic Finite Automaton (DFA) didefinisikan sebagai sebuah 5-tuple:

$$A = (Q, \Sigma, \delta, q_0, F)$$

di mana setiap komponen memiliki peran sebagai berikut:

1. Q adalah himpunan berhingga (finite) dari keadaan (states).
2. Σ adalah himpunan berhingga simbol masukan yang disebut alfabet.
3. δ adalah fungsi transisi yang memetakan pasangan (state, simbol masukan) ke sebuah state tujuan, ditulis $\delta : Q \times \Sigma \rightarrow Q$.
4. $q_0 \in Q$ adalah keadaan awal (start state).
5. $F \subseteq Q$ adalah himpunan keadaan penerima (accepting states atau final states).

Representasi paling umum dari DFA adalah diagram transisi dan tabel transisi. Pada diagram transisi, setiap state direpresentasikan sebagai simpul (node), setiap transisi direpresentasikan sebagai busur (arc) berlabel simbol masukan, start state ditandai dengan panah masuk, dan accepting states ditandai dengan lingkaran ganda. Pada tabel transisi, baris merepresentasikan state, kolom merepresentasikan simbol masukan, dan setiap sel berisi state tujuan δ(q, a).

1.5.2. Cara DFA Memproses String

DFA memproses sebuah string masukan $a_1a_2 \cdots a$□ dengan cara sebagai berikut. Dimulai dari start state q_0, DFA membaca simbol pertama a_1 dan melakukan transisi ke state $q_1 = \delta(q_0, a_1)$. Proses ini dilanjutkan secara berurutan sehingga diperoleh rangkaian state $q_0, q_1, q_2, \ldots, q$□ di mana $\delta(q_{i-1}, a_i) = q_i$ untuk setiap i. String tersebut

diterima jika dan hanya jika state akhir q□ merupakan anggota dari F (accepting states), dan ditolak apabila q□ ∉ F.

Konsep ini dapat diperluas dengan mendefinisikan fungsi transisi yang diperluas δ^* (extended transition function) yang memetakan pasangan (state, string) ke sebuah state. Fungsi ini didefinisikan secara induktif: untuk string kosong ε, $\delta^*(q, \varepsilon) = q$; dan untuk string w = xa, $\delta^*(q, w) = \delta(\delta^*(q, x), a)$. Dengan demikian, bahasa yang diterima DFA A, dinotasikan L(A), adalah:

$$L(A) = \{ w \mid \delta^*(q_0, w) \in F \}$$

Bahasa ini termasuk dalam kelas bahasa reguler. Setiap bahasa yang dapat diterima oleh suatu DFA disebut bahasa reguler (regular language).

1.5.3. Diagram Transisi dan Tabel Transisi

Terdapat dua cara utama untuk merepresentasikan DFA secara praktis. Cara pertama adalah diagram transisi. Sebuah diagram transisi untuk DFA $A = (Q, \Sigma, \delta, q_0, F)$ adalah graf berarah di mana:

- Setiap state $q \in Q$ direpresentasikan sebagai simpul dalam graf.
- Untuk setiap state $q \in Q$ dan simbol $a \in \Sigma$, jika $\delta(q, a) = p$, maka terdapat busur dari simpul q ke simpul p dengan label a. Apabila terdapat beberapa simbol yang menyebabkan transisi dari q ke p yang sama, simbol-simbol tersebut dapat digabungkan menjadi satu label pada satu busur.
- Terdapat sebuah panah masuk ke start state q_0 yang tidak berasal dari simpul mana pun.
- Simpul yang merepresentasikan accepting states (anggota F) ditandai dengan lingkaran ganda, sedangkan state biasa menggunakan lingkaran tunggal.

Cara kedua adalah tabel transisi. Tabel transisi adalah representasi tabular dari fungsi δ dengan baris merepresentasikan state dan kolom merepresentasikan simbol masukan. Setiap sel pada baris state q dan kolom simbol a berisi state δ(q, a). Start

state ditandai dengan tanda panah (→) dan accepting states ditandai dengan tanda bintang (*). Dengan melihat tabel transisi, seluruh informasi yang diperlukan untuk mendeskripsikan DFA secara unik dapat diperoleh.

1.6. Implementasi DFA dalam *Lexical Analysis*

Analisis leksikal (*lexical analysis*) merupakan tahap pertama dalam proses kompilasi. Komponen yang bertanggung jawab atas tahap ini disebut lexer atau *lexical analyzer*. Menurut Hopcroft, Motwani, dan Ullman (2006), salah satu aplikasi tertua dari ekspresi reguler adalah dalam menspesifikasi komponen compiler yang disebut "*lexical analyzer*." Komponen ini memindai program sumber dan mengenali semua token, yaitu substring dari karakter-karakter yang berurutan yang secara logis termasuk bersama-sama.

1.6.1. Peran DFA dalam Lexical Analysis

Setiap kelas token dalam bahasa pemrograman dapat dideskripsikan menggunakan ekspresi reguler. Karena setiap ekspresi reguler dapat dikonversi menjadi DFA yang menerima bahasa yang sama, lexer dapat diimplementasikan sebagai DFA. Hopcroft dkk. (2006) menjelaskan bahwa proses konversi dari kumpulan ekspresi reguler ke automaton dimulai dengan membangun automaton untuk gabungan (*union*) seluruh ekspresi. Automaton gabungan ini, ketika berada pada sebuah *accepting state*, secara tepat memberitahu token mana yang baru saja dikenali.

Dalam implementasinya, DFA untuk lexical analysis bekerja dengan prinsip berikut. DFA dimulai dari start state pada setiap awal token baru. Untuk setiap karakter yang dibaca dari *source code*, DFA melakukan transisi sesuai fungsi δ berdasarkan state saat ini dan karakter tersebut. Ketika DFA mencapai sebuah *accepting state*, sebuah token telah dikenali. Proses ini diulang dari awal untuk token berikutnya.

1.6.2. Penanganan Prioritas Token dan Kata Kunci

Salah satu tantangan dalam lexical analysis adalah penanganan konflik antara beberapa kelas token yang dapat mengenali string yang sama. Contoh paling umum adalah antara kata kunci (keyword) dan pengenal (identifier). String "begin"

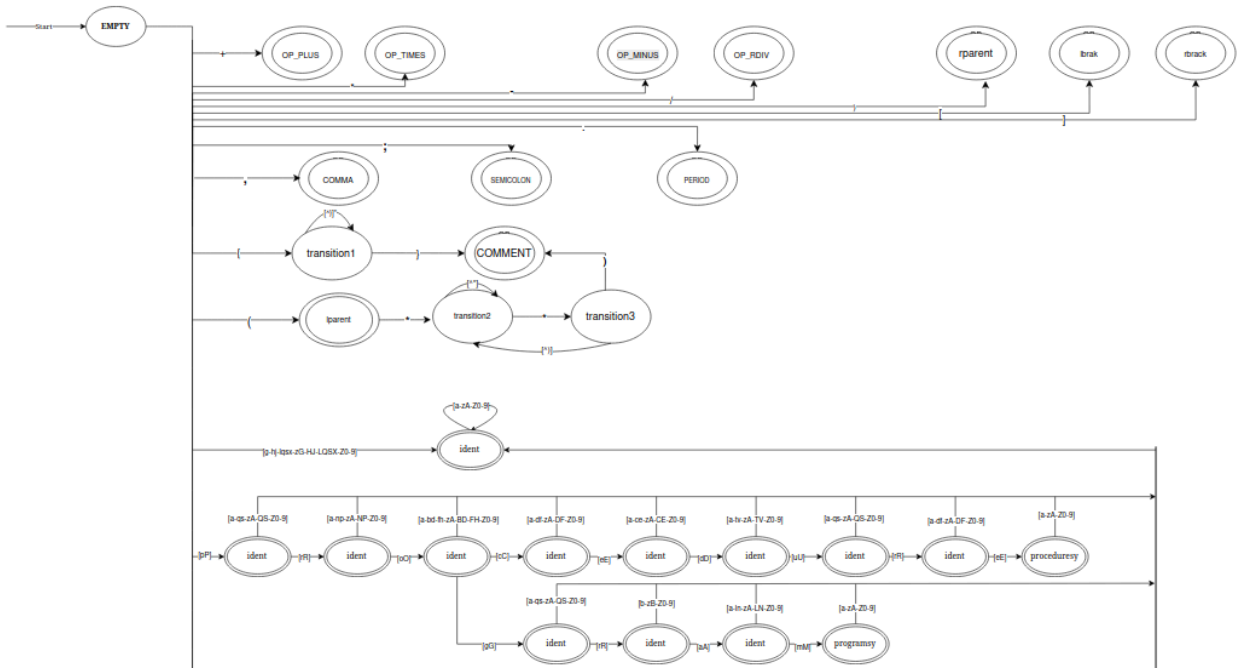
misalnya, secara sintaksis cocok dengan pola identifier (diawali huruf, diikuti huruf dan angka), tetapi juga merupakan kata kunci dalam bahasa Arion.

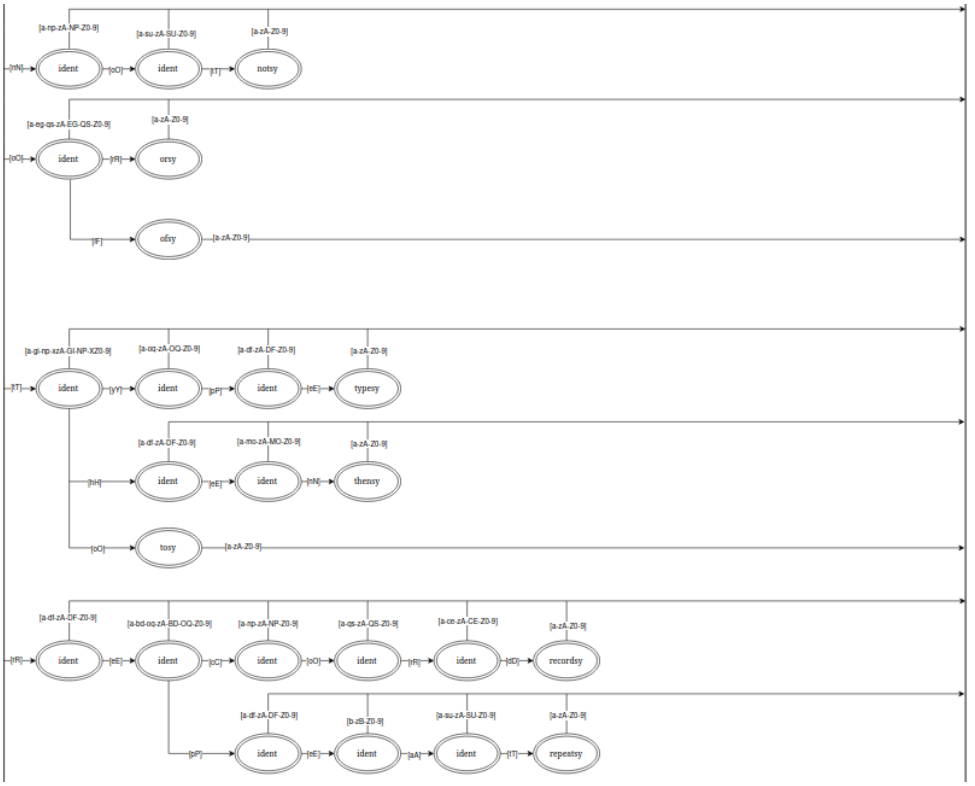
Hopcroft dkk. (2006) menjelaskan bahwa resolusi standar untuk kasus ini adalah memberikan prioritas pada ekspresi yang terdaftar lebih awal. Dalam implementasi lexer, hal ini diwujudkan melalui penggunaan symbol table atau keyword map: setelah DFA mengenali sebuah token dengan pola identifier, lexer memeriksa apakah string tersebut terdapat dalam tabel kata kunci. Jika ya, token dikembalikan dengan tipe kata kunci yang sesuai; jika tidak, token dikembalikan sebagai IDENT.

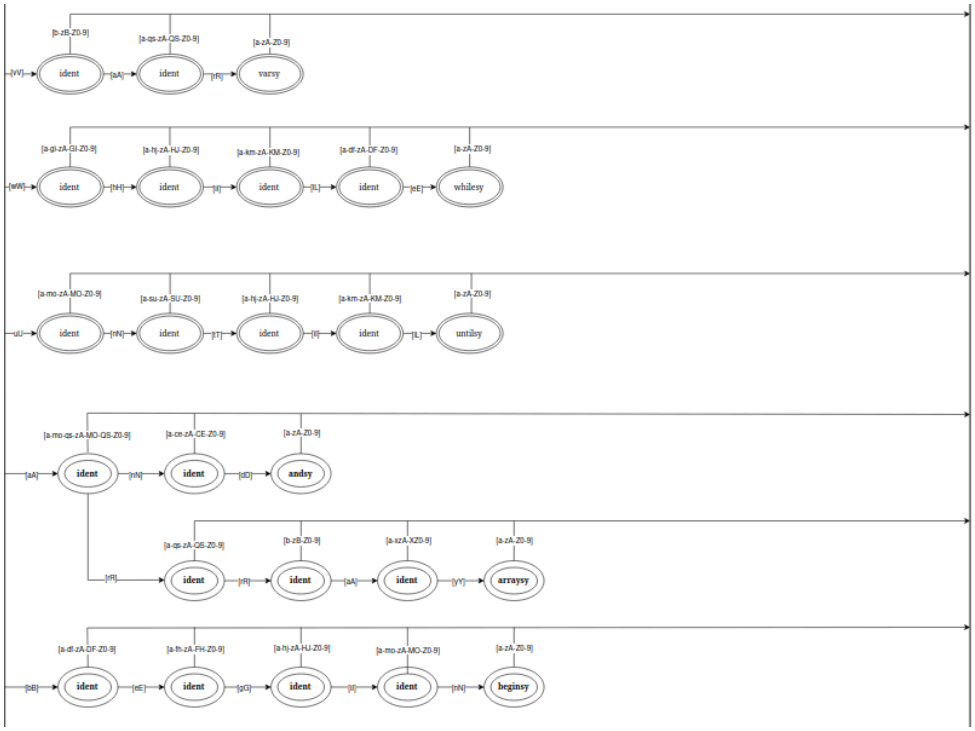
BAB 2
PERANCANGAN & IMPLEMENTASI

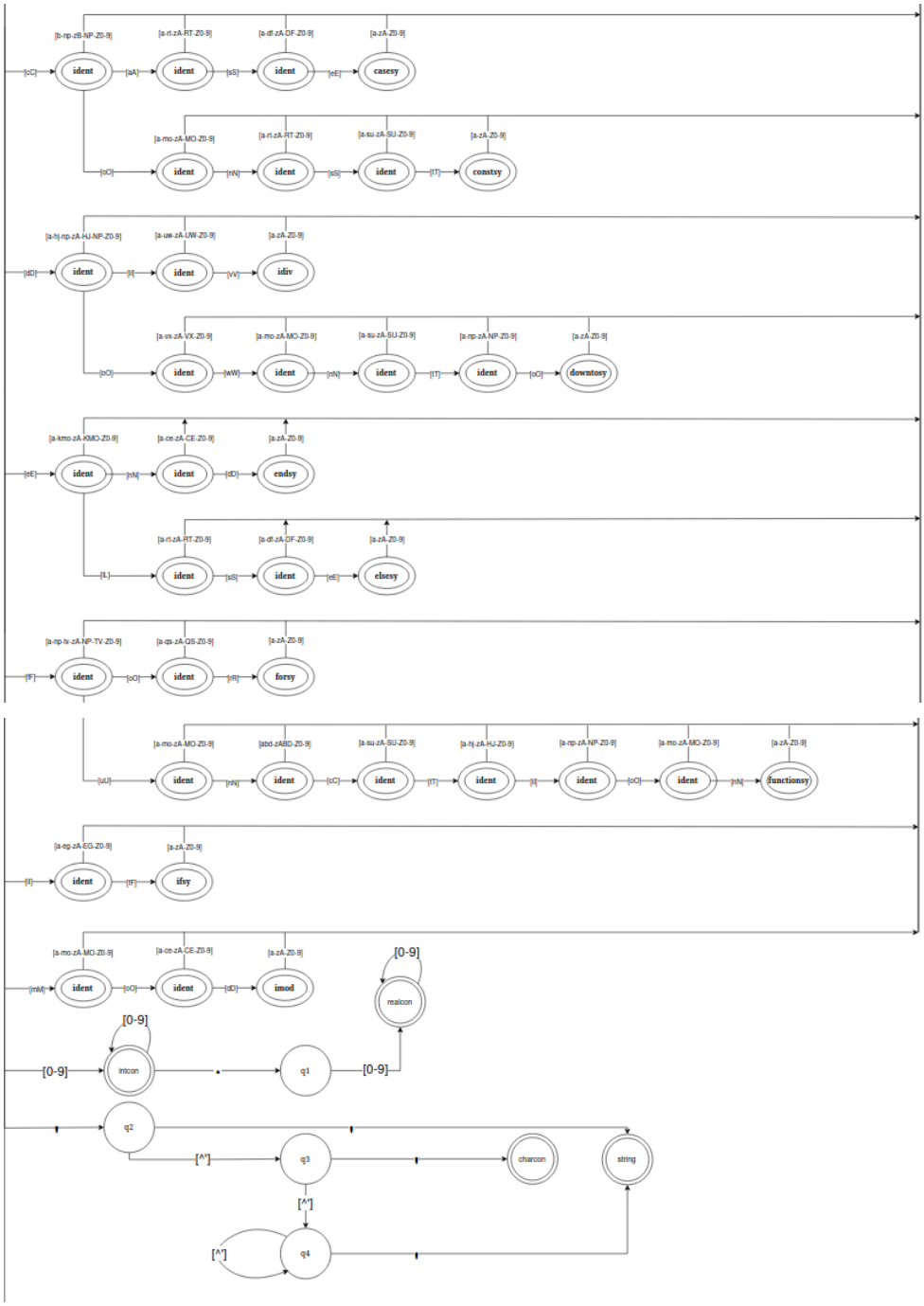
2.1. Perancangan

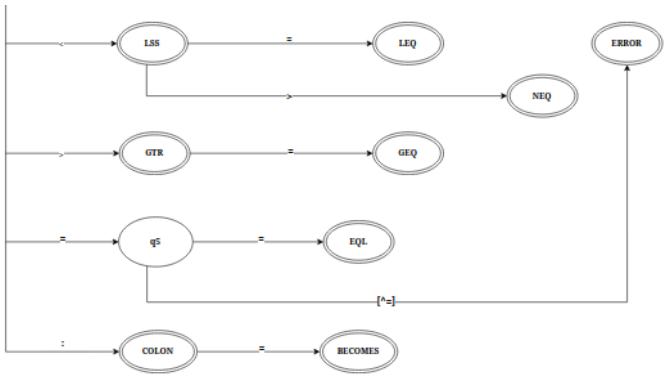
Dalam perancangan lexical analyzer untuk bahasa Arion, model komputasi yang dipilih adalah Deterministic Finite Automata (DFA). Pilihan ini didasarkan pada keunggulan teoretis yang melekat pada sifat DFA itu sendiri. Berbeda dengan NFA (Non-deterministic Finite Automata) yang memperbolehkan lebih dari satu transisi untuk pasangan (state, input) yang sama, DFA menjamin bahwa untuk setiap state aktif dan setiap simbol input yang dibaca, terdapat tepat satu state berikutnya yang dapat dituju. Dalam pembuatan DFA ini, kami melakukan *brainstorming* dengan membagi token-token ke dalam kategori tertentu untuk memudahkan pembagian tugas, membuat regex untuk menyamakan persepsi dari tiap token, lalu membuat diagram DFA. Hasil diagram DFA kami adalah sebagai berikut:




[a-np-zA-NP-Z0-9]
[a-su-zA-SU-Z0-9]
[a-zA-Z0-9]
[nN]
ident
[oO]
ident
[tT]
notsy
[a-eq-qs-zA-EG-QS-Z0-9]
[a-zA-Z0-9]
[rO]
ident
[rR]
orsy
[fF]
ofsy
[a-zA-Z0-9]
[a-gi-np-xzA-GI-NP-XZ0-9]
[a-oq-zA-OQ-Z0-9]
[a-df-zA-DF-Z0-9]
[a-zA-Z0-9]
[tT]
ident
[yY]
ident
[pP]
ident
[eE]
typesy
[a-df-zA-DF-Z0-9]
[a-mo-zA-MO-Z0-9]
[a-zA-Z0-9]
[hH]
ident
[eE]
ident
[nN]
thensy
[oO]
tosy
[a-zA-Z0-9]
[a-df-zA-DF-Z0-9]
[a-bd-oq-zA-BD-OQ-Z0-9]
[a-np-zA-NP-Z0-9]
[a-qs-zA-QS-Z0-9]
[a-cs-zA-CE-Z0-9]
[a-zA-Z0-9]
[rR]
ident
[eE]
ident
[cC]
ident
[oO]
ident
[rR]
ident
[dD]
recordsy
[a-df-zA-DF-Z0-9]
[b-zB-Z0-9]
[a-su-zA-SU-Z0-9]
[a-zA-Z0-9]
[pP]
ident
[eE]
ident
[aA]
ident
[tT]
repeatsy


[b-zB-Z0-9]
[a-qs-zA-QS-Z0-9]
[a-zA-Z0-9]
[vV]
ident
[aA]
ident
[rR]
varsy
[a-gi-zA-GI-Z0-9]
[a-hj-zA-HJ-Z0-9]
[a-km-zA-KM-Z0-9]
[a-df-zA-DF-Z0-9]
[a-zA-Z0-9]
[wW]
ident
[hH]
ident
[iI]
ident
[lL]
ident
[eE]
whilesy
[a-mo-zA-MO-Z0-9]
[a-su-zA-SU-Z0-9]
[a-hj-zA-HJ-Z0-9]
[a-km-zA-KM-Z0-9]
[a-zA-Z0-9]
uU
ident
[nN]
ident
[tT]
ident
[iI]
ident
[lL]
untilsy
[a-mo-qs-zA-MO-QS-Z0-9]
[a-cs-zA-CE-Z0-9]
[a-zA-Z0-9]
[aA]
ident
[nN]
ident
[dD]
andsy
[a-qs-zA-QS-Z0-9]
[b-zB-Z0-9]
[a-xzA-XZ0-9]
[a-zA-Z0-9]
[rR]
ident
[rR]
ident
[aA]
ident
[yY]
arraysy
[a-df-zA-DF-Z0-9]
[a-fh-zA-FH-Z0-9]
[a-hj-zA-HJ-Z0-9]
[a-mo-zA-MO-Z0-9]
[a-zA-Z0-9]
[bB]
ident
[eE]
ident
[gG]
ident
[iI]
ident
[nN]
beginsy


[b-np-zB-NP-Z0-9]
[a-rt-zA-RT-Z0-9]
[a-df-zA-DF-Z0-9]
[a-zA-Z0-9]
[cC]
ident
[aA]
ident
[sS]
ident
[eE]
casesy
[a-mo-zA-MO-Z0-9]
[a-rt-zA-RT-Z0-9]
[a-su-zA-SU-Z0-9]
[a-zA-Z0-9]
[oO]
ident
[nN]
ident
[sS]
ident
[tT]
constsy
[a-hj-np-zA-HJ-NP-Z0-9]
[a-uw-zA-UW-Z0-9]
[a-zA-Z0-9]
[dD]
ident
[iI]
ident
[vV]
idiv
[a-vx-zA-VX-Z0-9]
[a-mo-zA-MO-Z0-9]
[a-su-zA-SU-Z0-9]
[a-np-zA-NP-Z0-9]
[a-zA-Z0-9]
[oO]
ident
[wW]
ident
[nN]
ident
[tT]
ident
[oC]
downtosy
[a-kmo-zA-KMO-Z0-9]
[a-ce-zA-CE-Z0-9]
[a-zA-Z0-9]
[eE]
ident
[nN]
ident
[dD]
endsy
[a-rt-zA-RT-Z0-9]
[a-df-zA-DF-Z0-9]
[a-zA-Z0-9]
[lL]
ident
[sS]
ident
[eE]
elsesy
[a-np-tv-zA-NP-TV-Z0-9]
[a-qs-zA-QS-Z0-9]
[a-zA-Z0-9]
[fF]
ident
[oO]
ident
[rR]
forsy
[a-mo-zA-MO-Z0-9]
[abd-zABD-Z0-9]
[a-su-zA-SU-Z0-9]
[a-hj-zA-HJ-Z0-9]
[a-np-zA-NP-Z0-9]
[a-mo-zA-MO-Z0-9]
[a-zA-Z0-9]
[uU]
ident
[nN]
ident
[cC]
ident
[tT]
ident
[iI]
ident
[oO]
ident
[nN]
functionsy
[a-ep-zA-EG-Z0-9]
[a-zA-Z0-9]
[iI]
ident
[fF]
ifsy
[a-mo-zA-MO-Z0-9]
[a-ce-zA-CE-Z0-9]
[a-zA-Z0-9]
[mM]
ident
[oO]
ident
[dD]
imod
[0-9]
realcon
[0-9]
intcon
[0-9]
q1
[0-9]
q2
[^']
q3
charcon
string
[^']
[^']
q4



Gambar 1. Diagram DFA

Tabel 1. Keterangan State DFA

No.	Nama State	Deskripsi Transisi
1	**EMPTY**	Merupakan state awal (start state). DFA belum membaca simbol masukan apapun, atau baru saja menyelesaikan satu token dan siap membaca token berikutnya.
2	**OP_PLUS**	DFA membaca simbol '+' dari state EMPTY.
3	**OP_MINUS**	DFA membaca simbol '-' dari state EMPTY.
4	**OP_TIMES**	DFA membaca simbol '*' dari state EMPTY.
5	**OP_RDIV**	DFA membaca simbol '/' dari state EMPTY.
6	**rparent**	DFA membaca simbol ')' dari state EMPTY.
7	**lbrak**	DFA membaca simbol '[' dari state EMPTY.
8	**rbrack**	DFA membaca simbol ']' dari state EMPTY.

9	**COMMA**	DFA membaca simbol ',' dari state EMPTY.
10	**SEMICOLON**	DFA membaca simbol ';' dari state EMPTY.
11	**PERIOD**	DFA membaca simbol '.' dari state EMPTY.
12	**lparent**	DFA membaca simbol '(' dari state EMPTY. State ini bersifat antara: jika karakter berikutnya adalah '*', DFA masuk ke alur komentar '(* ... *)'; jika bukan, state ini diterima sebagai token lparent.
13	**LSS**	DFA membaca simbol '<' dari state EMPTY.
14	**LEQ**	DFA berada di state LSS kemudian membaca simbol '='.
15	**NEQ**	DFA berada di state LSS kemudian membaca simbol '>'.
16	**GTR**	DFA membaca simbol '>' dari state EMPTY.
17	**GEQ**	DFA berada di state GTR kemudian membaca simbol '='.
19	**EQL**	DFA berada di state q5 kemudian membaca simbol '=' kembali.
20	**COLON**	DFA membaca simbol ':' dari state EMPTY dan karakter berikutnya bukan '='.
21	**BECOMES**	DFA berada di state COLON kemudian membaca simbol '='.
23	**q1**	DFA berada di state intcon kemudian membaca simbol '.' (titik desimal). State ini adalah state antara yang menunggu digit desimal berikutnya.
25	**q2**	DFA membaca simbol tanda kutip tunggal "'" dari state EMPTY. State ini adalah awal pembacaan literal karakter atau string.

26	**q3**	DFA berada di state q2 kemudian membaca tepat satu karakter apapun selain tanda kutip tunggal ([^']). State ini menandai bahwa satu karakter telah terbaca setelah kutip pembuka.
27	**q4**	DFA berada di state q3 kemudian membaca satu atau lebih karakter selain tanda kutip ([^']). State ini memiliki self-loop pada input [^'], menandai dua atau lebih karakter dalam kutipan.
18	**q5**	DFA membaca simbol '=' dari state EMPTY. State ini merupakan state antara untuk membedakan antara operator '=' (EQL) dan ':=' (BECOMES).
22	**intcon**	DFA membaca satu atau lebih digit [0-9] secara berurutan dari state EMPTY. State ini memiliki self-loop pada input [0-9].
24	**realcon**	DFA berada di state q1 kemudian membaca satu atau lebih digit [0-9]. State ini memiliki self-loop pada input [0-9].
28	**charcon**	DFA berada di state q3 kemudian membaca tanda kutip penutup.
29	**string**	DFA berada di state q4 kemudian membaca tanda kutip penutup. Juga dapat dicapai dari q2 langsung dengan tanda kutip.
30	**transition1**	DFA membaca simbol '{' dari state EMPTY, memulai komentar bergaya {...}. State ini memiliki self-loop pada semua karakter kecuali '}' ([^}]*), mengonsumsi seluruh isi komentar.
31	**transition2**	DFA berada di state lparent kemudian membaca simbol '*', memulai komentar bergaya (* ... *). State ini memiliki self-loop pada semua karakter kecuali '*' ([^*]).
32	**transition3**	DFA berada di state transition2 kemudian membaca simbol '*'. State ini menunggu konfirmasi apakah karakter berikutnya adalah ')' (menutup komentar) atau bukan.
33	**COMMENT**	DFA berada di state transition1 kemudian membaca simbol '}', ATAU DFA berada di state transition3 kemudian membaca simbol ')'. Keduanya menandai akhir komentar.

34	**ERROR**	DFA mencapai state ini ketika membaca karakter yang tidak valid dalam konteks tertentu, misalnya membaca karakter selain '=' setelah '=' pada konteks operator perbandingan (q5 + [^=]).
36	**andsy**	DFA telah membaca urutan karakter [aA]→[nN]→[dD] secara berurutan dan tidak ada karakter [a-zA-Z0-9] setelahnya.
37	**arraysy**	DFA telah membaca urutan [aA]→[rR]→[rR]→[aA]→[yY] secara berurutan dan tidak ada karakter [a-zA-Z0-9] setelahnya.
38	**beginsy**	DFA telah membaca urutan [bB]→[eE]→[gG]→[iI]→[nN] secara berurutan dan tidak ada karakter [a-zA-Z0-9] setelahnya.
39	**casesy**	DFA telah membaca urutan [cC]→[aA]→[sS]→[eE] secara berurutan dan tidak ada karakter [a-zA-Z0-9] setelahnya.
40	**constsy**	DFA telah membaca urutan [cC]→[oO]→[nN]→[sS]→[tT] secara berurutan dan tidak ada karakter [a-zA-Z0-9] setelahnya.
41	**downtosy**	DFA telah membaca urutan [dD]→[oO]→[wW]→[nN]→[tT]→[oO] secara berurutan dan tidak ada karakter [a-zA-Z0-9] setelahnya.
42	**elsesy**	DFA telah membaca urutan [eE]→[lL]→[sS]→[eE] secara berurutan dan tidak ada karakter [a-zA-Z0-9] setelahnya.
43	**endsy**	DFA telah membaca urutan [eE]→[nN]→[dD] secara berurutan dan tidak ada karakter [a-zA-Z0-9] setelahnya.
44	**forsy**	DFA telah membaca urutan [fF]→[oO]→[rR] secara berurutan dan tidak ada karakter [a-zA-Z0-9] setelahnya.
45	**functionsy**	DFA telah membaca urutan [fF]→[uU]→[nN]→[cC]→[tT]→[iI]→[oO]→[nN] secara berurutan dan tidak ada karakter [a-zA-Z0-9] setelahnya.
46	**ifsy**	DFA telah membaca urutan [iI]→[fF] secara berurutan dan tidak ada karakter [a-zA-Z0-9] setelahnya.
47	**idiv**	DFA telah membaca urutan [iI]→[dD]→[iI]→[vV] secara berurutan dan tidak ada karakter [a-zA-Z0-9] setelahnya.
48	**imod**	DFA telah membaca urutan [iI]→[mM]→[oO]→[dD] secara berurutan dan tidak ada karakter [a-zA-Z0-9] setelahnya.

49	**notsy**	DFA telah membaca urutan [nN]→[oO]→[tT] secara berurutan dan tidak ada karakter [a-zA-Z0-9] setelahnya.
50	**ofsy**	DFA telah membaca urutan [oO]→[fF] secara berurutan dan tidak ada karakter [a-zA-Z0-9] setelahnya.
51	**orsy**	DFA telah membaca urutan [oO]→[rR] secara berurutan dan tidak ada karakter [a-zA-Z0-9] setelahnya.
52	**proceduresy**	DFA telah membaca urutan [pP]→[rR]→[oO]→[cC]→[eE]→[dD]→[uU]→[rR]→[eE] secara berurutan dan tidak ada karakter [a-zA-Z0-9] setelahnya.
53	**programsy**	DFA telah membaca urutan [pP]→[rR]→[oO]→[gG]→[rR]→[aA]→[mM] secara berurutan dan tidak ada karakter [a-zA-Z0-9] setelahnya.
54	**recordsy**	DFA telah membaca urutan [rR]→[eE]→[cC]→[oO]→[rR]→[dD] secara berurutan dan tidak ada karakter [a-zA-Z0-9] setelahnya.
55	**repeatsy**	DFA telah membaca urutan [rR]→[eE]→[pP]→[eE]→[aA]→[tT] secara berurutan dan tidak ada karakter [a-zA-Z0-9] setelahnya.
56	**thensy**	DFA telah membaca urutan [tT]→[hH]→[eE]→[nN] secara berurutan dan tidak ada karakter [a-zA-Z0-9] setelahnya.
57	**tosy**	DFA telah membaca urutan [tT]→[oO] secara berurutan dan tidak ada karakter [a-zA-Z0-9] setelahnya.
58	**typesy**	DFA telah membaca urutan [tT]→[yY]→[pP]→[eE] secara berurutan dan tidak ada karakter [a-zA-Z0-9] setelahnya.
59	**untilsy**	DFA telah membaca urutan [uU]→[nN]→[tT]→[iI]→[lL] secara berurutan dan tidak ada karakter [a-zA-Z0-9] setelahnya.
60	**varsy**	DFA telah membaca urutan [vV]→[aA]→[rR] secara berurutan dan tidak ada karakter [a-zA-Z0-9] setelahnya.
61	**whilesy**	DFA telah membaca urutan [wW]→[hH]→[iI]→[lL]→[eE] secara berurutan dan tidak ada karakter [a-zA-Z0-9] setelahnya.

62	**ident**	DFA membaca huruf pertama [a-zA-Z] yang bukan awalan khusus kata kunci manapun dari state EMPTY, ATAU huruf pertama yang merupakan awalan satu atau lebih kata kunci (mis. [aA] untuk 'and'/'array', [bB] untuk 'begin'). Semua state intermediate pada jalur pengenalan kata kunci berlabel 'ident' di diagram, menunjukkan bahwa input sementara masih dapat menjadi IDENTIFIER jika tidak membentuk kata kunci penuh, atau DFA telah membaca urutan karakter [a-zA-Z][a-zA-Z0-9]* yang TIDAK membentuk kata kunci manapun, ATAU sebuah awalan kata kunci yang dilanjutkan dengan karakter [a-zA-Z0-9] yang menyimpang dari pola kata kunci (mis. 'beginn', 'iff'). Setiap state intermediate keyword yang menerima [a-zA-Z0-9] di luar pola kata kunci akan kembali ke state ident ini.

2.2. Implementasi

2.2.1. Konsep Lexical Analyzer dan Peran DFA dalam Implementasi

Lexical analyzer (*lexer*) merupakan tahap awal dalam proses kompilasi yang bertugas membaca aliran karakter dari *source code* dan mengubahnya menjadi token yang merepresentasikan satuan bermakna. Dengan melakukan proses ini, *lexer* menyederhanakan struktur input sehingga dapat diproses lebih lanjut oleh tahap berikutnya dalam *compiler*. Secara teoritis, proses pengenalan token dimodelkan menggunakan *Deterministic Finite Automata* (DFA).

Dalam implementasi program, konsep tersebut direalisasikan melalui penggunaan kelas DFA sebagai mesin utama yang mengatur perpindahan *state* dan pembentukan token, khususnya melalui fungsi `nextToken()`. Sementara itu, kelas Lexer berfungsi sebagai antarmuka yang memanfaatkan DFA untuk melakukan tokenisasi secara keseluruhan. Dengan demikian, implementasi ini secara langsung merefleksikan konsep DFA yang telah dibahas pada Bab 1 ke dalam bentuk program.

2.2.2. Arsitektur dan Struktur Program

Program *lexical analyzer* ini dikembangkan menggunakan bahasa C++ yang didasarkan pada kapabilitas dalam performa eksekusi yang cepat, manipulasi string yang presisi, serta dukungan *Standard Template Library* (STL) yang efisien untuk implementasi struktur data. Kode sumber program ini dipisahkan ke dalam beberapa modul dan direktori yang memiliki *single responsibility*.

Arsitektur program dibagi menjadi komponen-komponen utama berikut:

- `main.cpp`

  Modul ini merupakan titik awal eksekusi program yang menangani argumen antarmuka baris perintah, mengolah pemanggilan modul-modul lain, dan mengatur pencetakan hasil akhir daftar token ke file keluaran.

- Modul `util`

  Modul ini terdiri dari `file.cpp` dan `file.hpp` yang bertanggung jawab untuk membaca isi file sumber dan menuliskan format akhir dari token yang dihasilkan ke file teks tujuan.

- Modul `token`

  Modul ini terdiri dari `token.cpp` dan `token.hpp` yang merepresentasikan struktur data dari token. Di dalamnya terdapat enumerasi `TokenType` yang memetakan jenis token dan kelas data `Token` yang menyimpan tipe token, nilai leksikal, dan posisi baris untuk keperluan pelaporan *error*.

- Modul `dfa`

  Modul ini terdiri dari `dfa.cpp` dan `dfa.hpp`. Kelas `DFA` beroperasi secara independen sebagai sebuah *state machine*. Modul ine bertugas untuk mengevaluasi input karakter satu per satu, melakukan transisi state, dan menghasilkan suatu objek `Token` setiap kali mencoba *accepting state*.

- Modul `lexer`

  Modul ini terdiri dari `lexer` dan `symbolTable`. Kelas `Lexer` mengekspos fungsi utama `tokenize()` yang akan melakukan perulangan pemanggilan token hingga menyentuh batas akhir file. Sementara itu, kelas `SymbolTable` memanfaatkan struktur *hash map* untuk membedakan antara *identifier* biasa dengan kata kunci.

2.2.3. Alur Kerja *Lexical Analyzer*

Proses *lexical analysis* dimulai dengan membaca file sumber yang kemudian diberikan ke objek `Lexer`. `Lexer` secara berulang memanggil fungsi `nextToken()` dari `DFA` untuk membaca karakter satu per satu dan menghasilkan token sesuai *state* dan aturan transisi. Token yang dihasilkan dikumpulkan hingga DFA mengembalikan token EOF sebagai penanda akhir input.

Pseudocode berikut merepresentasikan alur utama tokenisasi:

Pseudocode 1 Alur Perulangan Utama *Lexer*

```
function tokenize(source_code):
    tokens = empty list
    while not EOF:
        token = dfa.nextToken()
        if token.type is EOF: break
        tokens.append(token)
    return tokens
```

2.2.4. Implementasi DFA dan *State Transition*

Implementasi DFA dalam program direalisasikan melalui penggunaan enumerasi State yang merepresentasikan setiap state dalam automata, seperti `Q0`, `Q_ALPHA`, `Q_INT`, dan *state* lainnya. Proses analisis dimulai dari state awal (`Q0`) dan berlangsung dengan membaca karakter satu per satu menggunakan fungsi `advance()`. Setiap karakter yang dibaca akan menentukan perpindahan *state* berikutnya sesuai dengan aturan DFA yang telah dirancang.

Mekanisme transisi *state* diimplementasikan menggunakan struktur *switch-case* terhadap *state* aktif. Pada setiap iterasi, program akan mengevaluasi *state* saat ini dan karakter input untuk menentukan apakah tetap berada di *state* yang sama, berpindah ke *state* lain, atau menghasilkan token. *State* penerima ditandai dengan kondisi dengan token telah lengkap sehingga fungsi akan langsung mengembalikan objek token. Selain itu, digunakan mekanisme *backtracking* melalui fungsi `retract()` untuk mengembalikan satu karakter ke input ketika karakter tersebut bukan bagian dari token yang sedang diproses.

Berikut *pseudocode* yang merepresentasikan mekanisme utama DFA dalam fungsi `nextToken()`:

Pseudocode 2 Algoritma DFA untuk *Lexical Analysis*

```
function nextToken():
    skipWhitespace()
    if isEOF():
        return Token(KW_EOF, "", line)
    lexeme = ""
    currentState = Q0
    while true:
        // Baca 1 karakter dan majukan pointer
        c = advance()
        switch currentState:
            // State awal
            case Q0:
                if c == EOF:
                    return Token(KW_EOF, "", line)
                else if isAlpha(c):
                    lexeme += c
                    currentState = Q_ALPHA
                else if isDigit(c):
                    lexeme += c
                    currentState = Q_INT
                else if c == '\'':
                    currentState = Q_STR
                // ...(transisi awal untuk operator
multikarakter)
                else if isSingleCharOperator(c):
                    return Token(getOperatorType(c), c, line)
                else:
                    // Error handling karakter tidak dikenal
                    return Token(UNKNOWN, c, line)
```

```
            // State untuk Identifier
            case Q_ALPHA:
                if isAlphaNumeric(c):
                    lexeme += c
                else:
                    // Karakter saat ini milik token berikutnya
                    retract()
                    // Pengecekan apakah lexeme adalah Identifier
                    tokenType = SymbolTable.lookup(lexeme)
                    if tokenType is null:
                        tokenType = IDENTIFIER
                    return Token(tokenType, lexeme, line)

            // State untuk Konstanta Integer
            case Q_INT:
                if isDigit(c):
                    lexeme += c // Tetap di state Q_INT
                else if c == '.':
                    if isDigit(peek()):
                        lexeme += c
                        currentState = Q_REAL
                    else:
                        retract()
                        return Token(INTCON, lexeme, line)
                else:
                    retract()
                    return Token(INTCON, lexeme, line)

// ... (State lain seperti Q_REAL, Q_STR, Q_COLON, dll mematuhi
pola yang sama)
```

Pseudocode tersebut menunjukkan bagaimana DFA bekerja sebagai mesin deterministik yang mengatur alur pembacaan karakter, perpindahan *state*, serta pembentukan token. Setiap transisi yang terjadi dalam kode secara langsung merepresentasikan diagram DFA yang telah dirancang pada tahap perancangan.

2.2.5. Implementasi Pengenalan Token dan Optimasi *Symbol Table*

Pengenalan berbagai jenis token dalam bahasa Arion ditangani oleh DFA melalui pengelompokan pola karakter. Untuk menjaga efisiensi dan kebersihan kode, pengenalan token dibagi menjadi beberapa mekanisme utama:

- Pengenalan *Identifier* dan *Keyword*

  Identifier dan *keyword* diproses melalui state `Q_ALPHA`. Karakter huruf pertama akan membawa DFA ke *state* ini, lalu program terus membaca karakter alfanumerik sampai sebuah *lexeme* terbentuk. Setelah itu, *lexeme* tidak langsung dianggap sebagai *identifier*. Keputusan apakah *lexeme* tersebut merupakan *reserved word* dipisahkan dari logika DFA utama dan didelegasikan pada struktur *hash map* di dalam kelas `SymbolTable`.
- Pengenalan Literal Numerik

  DFA menggunakan state `Q_INT` untuk membaca deretan digit. Jika setelah integer ditemukan karakter titik, DFA akan melakukan *lookahead* menggunakan fungsi `peek()`. Jika karakter setelah titik adalah digit, state berpindah ke `Q_REAL`. Namun, jika bukan digit, titik tersebut akan dikembalikan ke pita input menggunakan fungsi `retract()`, dan hasilnya tetap diklasifikasikan sebagai intcon.
- Pengenalan *String* dan Karakter

  Literal bertanda kutip tunggal diproses pada state `Q_STR` lalu dibedakan berdasarkan panjang isi *lexeme*. Satu karakter murni diklasifikasikan sebagai `charcon`, sedangkan kosong atau lebih dari satu karakter diklasifikasikan sebagai `string`.
- Pengenalan Operator, Delimiter, dan Komentar

  Operator seperti +, -, :=, <=, <>, dan == dikenali melalui kombinasi state awal dan state transisi seperti `Q_COLON`, `Q_LT`, `Q_GT`, dan `Q_EQL`. Komentar {...} maupun (*...*) juga diproses melalui *state* khusus.

Optimasi utama pada program ini adalah memisahkan logika pengecekan *keyword* dari *state machine* DFA. Pendekatan ini membuat transisi DFA lebih sederhana dan menghindari kebutuhan untuk membuat ratusan state terpisah hanya untuk mengeja setiap *keyword* satu per satu. Sebelum pengecekan dilakukan melalui `symTable.lookup()`, semua *lexeme* diubah ke huruf kecil terlebih dahulu sehingga pengecekan bersifat *case-insensitive* dengan pencarian waktu konstan $O(1)$.

Pseudocode 3 Algoritma Resolusi *Identifier* dan *Keyword* menggunakan *Symbol Table*

```
function resolveAlphaToken(lexeme):
    // Ubah lexeme menjadi huruf kecil agar case-insensitive
    lower_lexeme = toLowerCase(lexeme)

    // Lookup ke dalam Hash Map (kompleksitas O(1))
    if lower_lexeme exists in SymbolTable.keywords:
        tokenType = SymbolTable.keywords.get(lower_lexeme)
        return tokenType
    else:
    // Jika tidak ada di daftar keyword, maka ia adalah
Identifier
        return IDENTIFIER
```

2.2.6. Penanganan Error

Program dirancang agar *robust* dan tidak berhenti secara tiba-tiba ketika menemukan input yang tidak valid. Pada kondisi masukan melanggar aturan leksikal, *lexer* akan menangkap kesalahan tersebut dan mengembalikan token bertipe `TokenType::UNKNOWN`. Dengan pendekatan ini, proses analisis akan tetap dilanjutkan hingga akhir file dan kesalahan dapat dilaporkan secara eksplisit pada output token. Penanganan error ini diimplementasikan langsung di dalam fungsi `nextToken()` melalui beberapa skenario berikut:

- Karakter Tidak Dikenal pada *State* Awal (`Q0`)

  Jika pada *state* awal ditemukan karakter yang tidak termasuk huruf, digit, tanda kutip, operator, delimiter, maupun awal komentar, program akan segera menghasilkan token `UNKNOWN` dengan nilai karakter tersebut.

- Operator Tidak Lengkap

  Penanganan error juga diterapkan pada operator = tunggal. Karena spesifikasi bahasa Arion hanya menerima == sebagai token eql dan := untuk becomes, simbol = yang berdiri sendiri tanpa pasangan akan langsung diklasifikasikan sebagai `UNKNOWN`.
- Literal *String* Tidak Ditutup

  Pada state `Q_STR`, *lexer* mengantisipasi literal yang tidak memiliki tanda kutip penutup. Jika lexer menemui akhir file atau mendeteksi baris baru sebelum tanda kutip penutup ditemukan, literal tersebut akan langsung dievaluasi dan dikembalikan sebagai `UNKNOWN`.
- Komentar Tidak Ditutup

  Hal yang sama berlaku pada komentar blok {...} dan (*...*). Jika penutup komentar tidak ditemukan hingga lexer menyentuh akhir file, program akan menghasilkan token `UNKNOWN` yang berisi *lexeme* komentar yang belum selesai tersebut.

Seluruh hasil *error* dari skenario di atas kemudian diformat oleh kelas `Token` menggunakan fungsi `toString()`, yang akan menghasilkan output dengan format *error*.

2.2.7. Representasi dan Output Token

Setiap hasil analisis leksikal direpresentasikan dalam bentuk objek dari kelas `Token`. Kelas ini memiliki tiga atribut utama, yaitu tipe token (*TokenType*), nilai token (*value*), dan nomor baris (*line*). Atribut tipe menunjukkan kategori token seperti `ident`, `intcon`, atau `plus`, sedangkan nilai menyimpan *lexeme* asli dari *source code*, misalnya nama variabel atau angka. Informasi nomor baris digunakan untuk memberikan konteks lokasi token dalam *source code*.

Untuk memastikan format output sesuai dengan spesifikasi, kelas `Token` menyediakan fungsi `tokenTypeName()` yang memetakan setiap `TokenType` ke representasi string yang diharapkan, seperti "intcon", "ident", atau "beginsy". Selain itu, fungsi `toString()` digunakan untuk menghasilkan format akhir token. Token yang memiliki nilai, seperti identifier, literal, comment, dan error, dicetak dalam format tipe (*value*), sedangkan token lain cukup dicetak sebagai tipe saja.

Dengan pendekatan ini, seluruh token yang dihasilkan oleh *lexer* dapat dicetak secara konsisten sesuai dengan format yang diminta sehingga output program mudah dibaca dan siap digunakan untuk tahap selanjutnya dalam proses kompilasi.

2.2.8. Integrasi Input dan Output Program

Program *lexer* diintegrasikan dengan mekanisme input-output berbasis *command line interface* (CLI). Program dijalankan dengan format `./lexer [input.txt] [output.txt]`, dengan file pertama merupakan *source code* yang akan dianalisis, sedangkan file kedua bersifat opsional sebagai tempat penyimpanan hasil tokenisasi. Program akan membaca file input, melakukan proses *lexical analysis*, lalu menampilkan hasil token ke terminal. Jika parameter output diberikan, hasil token juga akan disimpan ke file menggunakan fungsi `FileUtil::writeFile()`.

BAB 3
PENGUJIAN

Tabel 2. Pengujian program

No	Input	Output

1	Kasus: Masukan *valid* berupa program Arion sederhana dengan deklarasi variabel, *assignment*, operasi aritmatika, dan pemanggilan fungsi.
	`program Hello;`
	
	`var`
	`  a, b: integer;`
	
	`begin`
	`  a := 5;`
	`  b := a + 10;`
	`  writeln('Result = ', b);`
	`end.`
		`programsy`
		`ident (Hello)`
		`semicolon`
		`varsy`
		`ident (a)`
		`comma`
		`ident (b)`
		`colon`
		`ident (integer)`
		`semicolon`
		`beginsy`
		`ident (a)`
		`becomes`
		`intcon (5)`
		`semicolon`
		`ident (b)`
		`becomes`
		`ident (a)`
		`plus`
		`intcon (10)`
		`semicolon`
		`ident (writeln)`
		`lparent`
		`string ('Result = ')`
		`comma`
		`ident (b)`
		`rparent`
		`semicolon`
		`endsy`
		`period`
		
		Program berhasil ditokenisasi dengan benar. *Lexer* mengenali *programsy, varsy, beginsy,* dan *endsy* sebagai *keyword*, variabel a dan b sebagai *ident*, operator *:=* sebagai *becomes*, operasi + sebagai *plus*, bilangan sebagai *intcon*, dan *string 'Result = '* sebagai *string*.

<table>
<tr>
<td>2</td>
<td>Kasus: Masukan invalid yang mengandung karakter tidak dikenal dan string yang tidak ditutup.

```
program Bad;
var
  @illegal: integer;
  x: integer;
begin
  x := 'unclosed string;
  x := 5 $ 3;
end.
```
</td>
<td>

```
programsy
ident (Bad)
semicolon
varsy
error (@)
ident (illegal)
colon
ident (integer)
semicolon
ident (x)
colon
ident (integer)
semicolon
beginsy
ident (x)
becomes
error ('unclosed string;
)
ident (x)
becomes
intcon (5)
error ($)
intcon (3)
semicolon
endsy
period
```

Lexer berhasil menangani karakter tidak dikenal tanpa *crash*. Karakter @ dan *$* dikeluarkan sebagai *error (@)* dan *error ($)*. Selain itu, *string* yang tidak ditutup seperti *'unclosed string;* dikeluarkan sebagai *error* karena tidak ditemukan tanda kutip penutup sebelum akhir baris.
</td>
</tr>
</table>

<table>
<tr>
<td>3</td>
<td>Kasus: Masukan valid dengan keyword ditulis dalam campuran uppercase dan lowercase.

```
PROGRAM CaseInsensitive;

VAR
  a, b: integer;
BEGIN
  a := 5;
  b := 10;
  IF a == b then
    a := 1
  ELSE
    b := 2;
  while a > 0 DO
    a := a - 1;
  FOR b := 1 TO 5 do
    b := b + 1;
END.
```
</td>
<td>

```
programsy
ident (CaseInsensitive)
semicolon
varsy
ident (a)
comma
ident (b)
colon
ident (integer)
semicolon
beginsy
ident (a)
becomes
intcon (5)
semicolon
ident (b)
becomes
intcon (10)
semicolon
ifsy
ident (a)
eql
ident (b)
thensy
ident (a)
becomes
intcon (1)
elsesy
ident (b)
becomes
intcon (2)
semicolon
whilesy
ident (a)
gtr
intcon (0)
dosy
ident (a)
becomes
ident (a)
minus
intcon (1)
semicolon
forsy
ident (b)
becomes
intcon (1)
tosy
intcon (5)
dosy
ident (b)
becomes
ident (b)
plus
intcon (1)
semicolon
```

```
endsy
period
```

Lexer berhasil mengenali *keyword* yang ditulis dalam campuran *uppercase* dan *lowercase*.
</td>
</tr>
</table>

<table>
<tr>
<td>4</td>
<td>Kasus: Masukan valid yang menguji edge case seperti komentar brace { } dan paren (* *), konstanta real, konstanta karakter, operator relasional, dan perulangan downto.

```
program EdgeCases;

{ komentar brace }
(* komentar paren *)

var
  pi   : real;
  ch   : char;

begin
  pi   := 3.14;
  ch   := 'z';

  if 5 <> 10 then
    pi := NOT pi;

  if pi >= 3.0 then
    pi := pi + 26.7;

  (* komentar
     multiline *)

  for pi := 1 downto 0 do
    pi := pi + 1;
end.
```

```
programsy
ident (EdgeCases)
semicolon
comment ({ komentar brace })
comment ((* komentar paren *))
varsy
ident (pi)
colon
ident (real)
semicolon
ident (ch)
colon
ident (char)
semicolon
beginsy
ident (pi)
becomes
realcon (3.14)
semicolon
ident (ch)
becomes
charcon ('z')
semicolon
ifsy
intcon (5)
neq
intcon (10)
thensy
ident (pi)
becomes
notsy
ident (pi)
semicolon
ifsy
ident (pi)
geq
realcon (3.0)
thensy
ident (pi)
becomes
ident (pi)
plus
realcon (26.7)
semicolon
comment ((* komentar

    multiline *))
forsy
ident (pi)
becomes
intcon (1)
downtosy
intcon (0)
dosy
ident (pi)
becomes
```

```
ident (pi)
plus
intcon (1)
semicolon
endsy
period
```

Lexer berhasil menangani berbagai *edge*</td>
</tr>
<tr>
<td></td>
<td></td>
<td></td>
</tr>
</table>

<table>
<tr><td></td><td></td><td>case. Komentar { } dan (* *) termasuk multiline dikenali sebagai comment. Konstanta real 3.14 dan 26.7 dikenali sebagai realcon. Konstanta karakter ‘z’ dikenali sebagai charcon. Operator relasional <> dan >= masing-masing dikenali sebagai neq dan geq. Keyword downto dikenali sebagai downtosy.</td></tr>
<tr><td>5</td><td>Kasus: Masukan valid yang menguji kemampuan lexer untuk membedakan tipe literal, yaitu charcon, string, dan realcon.

```
program LiteralTypes;
var
  c: char;
  s: string;
  x, y: real;
  a: integer;
begin
  { charcon: tepat 1 karakter }
  c := 'z';
  c := 'a';

  { string: lebih dari 1 karakter }
  s := 'hello';
  s := 'IRK';

  { realcon: angka dengan titik desimal diikuti digit }
  x := 3.14;
  y := 26.7;

  { intcon + period: angka diikuti titik tanpa digit setelahnya }
  a := 5;
  x := x + y;
end.
```

```
programsy
ident (LiteralTypes)
semicolon
varsy
ident (c)
colon
ident (char)
semicolon
ident (s)
colon
ident (string)
semicolon
ident (x)
comma
ident (y)
colon
ident (real)
semicolon
ident (a)
colon
ident (integer)
semicolon
beginsy
comment ({ charcon: tepat 1 karakter })
ident (c)
becomes
charcon ('z')
semicolon
ident (c)
becomes
charcon ('a')
semicolon
comment ({ string: lebih dari 1 karakter })
ident (s)
becomes
string ('hello')
semicolon
ident (s)
becomes
string ('IRK')
semicolon
comment ({ realcon: angka dengan titik desimal diikuti digit })
ident (x)
becomes
realcon (3.14)
semicolon
ident (y)
becomes
realcon (26.7)
semicolon
comment ({ intcon + period: angka diikuti titik tanpa digit setelahnya })
ident (a)
becomes
intcon (5)
semicolon
ident (x)
```

```
becomes
ident (x)
plus
ident (y)
semicolon
endsy
period
```

Lexer berhasil membedakan tipe literal dengan bentuk yang serupa. ‘z’ dan ‘a’</td></tr>
</table>

<table>
<tr><td></td><td></td><td>dikenali sebagai charcon karena mengandung tepat 1 karakter, sedangkan ‘hello’ dan ‘IRK’ dikenali sebagai string karena mengandung lebih dari 1 karakter. Konstanta real 3.14 dan 26.7 dikenali sebagai realcon.</td></tr>
<tr><td>6</td><td>Kasus: Masukan mengandung operator == (valid) dan = (invalid).

```
program EqlTest;
var
  a, b: integer;
begin
  a := 5;
  b := 10;
  if a == b then
    a := 1;
  if a = b then
    a := 2;
end.
```

</td><td>

```
programsy
ident (EqlTest)
semicolon
varsy
ident (a)
comma
ident (b)
colon
ident (integer)
semicolon
beginsy
ident (a)
becomes
intcon (5)
semicolon
ident (b)
becomes
intcon (10)
semicolon
ifsy
ident (a)
eql
ident (b)
thensy
ident (a)
becomes
intcon (1)
semicolon
ifsy
ident (a)
error (=)
ident (b)
thensy
ident (a)
becomes
intcon (2)
semicolon
endsy
period
```

Lexer berhasil membedakan == dan = tunggal. == dikenali sebagai eql sedangkan = tunggal dikenali sebagai error (=) karena bukan token yang valid dalam bahasa Arion.</td></tr>
</table>

<table>
<tr>
<td>7</td>
<td>Kasus: Masukan yang menguji ambiguitas karakter dalam konteks komentar dan literal, seperti (di dalam { } yang tidak memutus komentar brace, serta { di dalam ( ) yang tidak memutus komentar paren. Selain itu, diuji pula string dengan jumlah tanda kutip genap dan ganjil.

```
program commentAmbiguity;
var
  x: integer;
  s: string;

begin
  { komentar brace dengan (* di dalamnya, bukan komentar paren }
  (* komentar paren dengan { di dalamnya, bukan komentar brace *)

  { komentar dengan karakter khusus: := + - * / < > = }

  (* komentar dengan bintang di tengah: 5 * 3 = 15 *)

  s := '''''''''''''''''''''';
  s := '''''''''''''''''''''';

  { operasi perkalian setelah lparent: jangan bingung dengan komentar }
  x := (5 * 3);
  x := (x * 2);
end.
```

```
programsy
ident (commentAmbiguity)
semicolon
varsy
ident (x)
colon
ident (integer)
semicolon
ident (s)
colon
ident (string)
semicolon
beginsy
comment ({ komentar brace dengan (* di dalamnya, bukan komentar paren })
comment ((* komentar paren dengan { di dalamnya, bukan komentar brace *))
comment ({ komentar dengan karakter khusus: := + - * / < > = })
comment ((* komentar dengan bintang di tengah: 5 * 3 = 15 *))
ident (s)
becomes
string ('')
string ('')
string ('')
string ('')
string ('')
string ('')
string ('')
string ('')
string ('')
string ('')
string ('')
semicolon
ident (s)
becomes
string ('')
string ('')
string ('')
string ('')
string ('')
string ('')
string ('')
string ('')
string ('')
string ('')
string ('')
string ('')
error (';
)
```

```
comment ({ operasi perkalian setelah lparent: jangan bingung dengan komentar })
ident (x)
becomes
lparent
intcon (5)
times
intcon (3)
rparent
semicolon
ident (x)
becomes
lparent
ident (x)
times
intcon (2)
rparent
semicolon
endsy
period
```

Program mengenali keempat komentar sebagai *comment* tanpa terpengaruh karakter khusus di dalamnya. `''''''''''''''''''''''` (22 tanda kutip = 11 pasang) menghasilkan 11 token *string(' ')* diikuti *error(' ';)* karena tanda kutip terakhir tidak menemukan kutip penutup sebelum ; yang menandakan akhir baris. `(5 * 3)` dan `(x * 2)` dikenali</td>
</tr>
</table>

<table>
<tr><td></td><td></td><td>dengan benar sebagai lparent dan operasi perkalian, bukan komentar.</td></tr>
<tr><td>8</td><td>Kasus: Masukan valid yang menguji batas antara keyword dan identifier, seperti identifier begin2, iff, dan forloop yang serupa dengan keyword harus tetap dikenali sebagai ident

```
program KwIdBoundary;
var
  begin2, iff, forloop: integer;
  score: integer;

begin
  { identifier yang mirip keyword harus tetap jadi ident }
  begin2  := 1;
  iff     := 2;
  forloop := 3;

  { case/of }
  score := 85;
  case score of
    100: begin2 := 1;
    85 : begin2 := 2;
    0  : begin2 := 0;
  end;

  { repeat/until }
  repeat
    forloop := forloop - 1;
  until forloop == 0;
end.
```

</td><td>

```
programsy
ident (KwIdBoundary)
semicolon
varsy
ident (begin2)
comma
ident (iff)
comma
ident (forloop)
colon
ident (integer)
semicolon
ident (score)
colon
ident (integer)
semicolon
beginsy
comment ({ identifier yang mirip keyword harus tetap jadi ident })
ident (begin2)
becomes
intcon (1)
semicolon
ident (iff)
becomes
intcon (2)
semicolon
ident (forloop)
becomes
intcon (3)
semicolon
comment ({ case/of })
ident (score)
becomes
intcon (85)
semicolon
casesy
ident (score)
ofsy
intcon (100)
colon
ident (begin2)
becomes
intcon (1)
semicolon
intcon (85)
colon
```

```
ident (begin2)
becomes
intcon (2)
semicolon
intcon (0)
colon
ident (begin2)
becomes
intcon (0)
semicolon
endsy
semicolon
comment ({ repeat/until })
repeatsy
ident (forloop)
becomes
ident (forloop)
minus
intcon (1)
semicolon
untilsy
ident (forloop)
eql
intcon (0)
semicolon
endsy
period
```

begin2, *iff*, dan *forloop* berhasil dikenali</td></tr>
</table>

		sebagai *ident* meskipun mengandung *substring keyword*, seperti *begin, if*, dan *for*, membuktikan bahwa DFA tidak berhenti di tengah *lexeme*. Selain itu, *casesy, ofsy, repeatsy,* dan *untilsy* dikenali dengan benar.

BAB 4
KESIMPULAN DAN SARAN

4.1 Kesimpulan

Program *lexical analyzer* untuk bahasa pemrograman Arion yang diimplementasikan menggunakan bahasa C++ telah berhasil direalisasikan dan dijalankan dengan baik untuk melakukan analisis leksikal terhadap *file* masukan. Program mampu membaca aliran karakter secara sekuensial dan mengidentifikasi token-token yang *valid* sesuai dengan aturan yang dimodelkan menggunakan *Deterministic Finite Automata* (DFA). Implementasi *lexer* dilakukan melalui secara manual tanpa memanfaatkan *library generator lexer* otomatis, sehingga seluruh mekanisme transisi *state*, pengenalan pola, dan validasi token merepresentasikan penerapan langsung dari konsep teori bahasa formal dan otomata. Program menerima masukan berupa *file* kode sumber dan memprosesnya menjadi daftar token yang telah tervalidasi dalam format *file* teks. Apabila seluruh isi *file* masukan *valid*, maka program akan menghasilkan daftar token yang lengkap dan telah diidentifikasi. Apabila ditemukan token yang *invalid*, maka bagian yang *invalid* akan diklasifikasikan sebagai *error* dan program akan melanjutkan proses analisis hingga akhir *file*. Dengan demikian, keluaran program dapat berupa kombinasi antara token yang *valid* dan penanda token *invalid*.

4.2 Saran

Program *lexical analyzer* yang dikembangkan telah berfungsi dengan baik dalam melakukan analisis token terhadap berkas masukan berdasarkan aturan leksikal yang dimodelkan menggunakan DFA. Program ini masih dapat dikembangkan lebih lanjut, misalnya dengan memperluas cakupan aturan token atau mengadaptasi model DFA untuk bahasa pemrograman lain untuk menguji fleksibilitas dan batas kemampuan sistem. Selain itu, pengujian program disarankan untuk diperluas dengan mencakup lebih banyak variasi kasus, termasuk kasus batas dan masukan *invalid*, untuk meningkatkan ketahanan program serta memastikan keandalan dalam berbagai kondisi masukan.

Tugas besar IF2224 Teori Bahasa Formal dan Otomata Milestone 1 ini telah membantu pengembang untuk memahami konsep *Deterministic Finite Automata* (DFA)

secara lebih mendalam melalui implementasi langsung dalam bentuk program *lexical analyzer*. Proses perancangan *state*, transisi, dan penanganan berbagai pola token memberikan pengalaman praktis yang memperkuat pemahaman teori yang sebelumnya dipelajari secara kontekstual.

BAB 5
LAMPIRAN

5.1 Link Repository Github

https://github.com/samuelsondhrm/JBC-Tubes-IF2224-2026

5.2 Link Workspace Diagram Transisi DFA

https://drive.google.com/file/d/1_r1o9q-rQX41qPOX3xguAGgH27M5R30n/view?usp=sharing

5.3 Pembagian Tugas

Tabel 3. Pembagian tugas

NIM	Pembagian tugas	Persentase
13524001	1. Menyusun teori singkat seputar DFA dan implementasi DFA dalam *Lexer making*. 2. Membuat *class* DFA pada program. 3. Merangkai DFA model pada bagian pembacaan *intcon*, *realcon*, *charcon*, *string*.	20%
13524027	1. Menyusun teori singkat mengenai *lexical analysis, regular expression, token recognition,* dan *finite state machine*. 2. Merangkai DFA model pada *logical operators, parantheses, brackets,* dan *comment*. 3. Mengimplementasikan I/O *file*, *string parsing*, dan class token.	20%
13524029	1. Menyusun sub bab implementasi pada bab perancangan dan implementasi.	20%

	2. Mengimplementasikan class lexer pada program. 3. Merangkai DFA model pada bagian pembacaan simbol *ident, proceduresy, programsy, notsy, orsy, ofsy, typesy, thensy, tosy, recordsy, repeatsy, varsy, whilesy,* dan *untilsy.*	
13524089	1. Menyusun dan menulis bab pengujian serta bab kesimpulan dan saran. 2. Mengimplementasikan *symbol table* pada program. 3. Merangkai DFA model pada bagian pembacaan simbol *lss (<), leq (<=), neq (<>), gtr (>), geq (>=), eql (==), becomes (:=),* dan *colon (:)*.	20%
13524093	1. Menyusun sub bab perancangan pada bab perancangan dan implementasi 2. Membuat *class* DFA pada program 3. Merangkai DFA model pada bagian pembacaan *andsy, arraysy, beginsy, casesy, constsy, downtosy, elsesy, endsy, forsy, functionsy, ifsy, idiv,* dan *imod*.	20%

5.4 Referensi

Aho, Alfred V, et al. *Compilers : Principles, Techniques, and Tools*. Noida, India, Dorling Kindersley (India) Pvt. Ltd., Licensees Of Pearson Education In South Asia, 2014.

"Compiler Design - Lexical Analysis - Tutorialspoint." *Tutorialspoint.com*, 2020, www.tutorialspoint.com/compiler_design/compiler_design_lexical_analysis.htm.